RÉCAPITULATIF DU PROJET

PokerVitrine

Plateforme communautaire poker francophone — Symfony + Angular + PostgreSQL

État au 29 octobre 2026
Dépôt : github.com/redkards/poker-vitrine
Dossier local : Bureau\site poker test

3 / 6	19	8	0
sprints du MVP terminés (0, 1, 2)	tests backend qui passent	commits sauvegardés sur GitHub	secret versionné (clés, mots de passe)

1. Où en est le projet

Les fondations, l'authentification complète et le contenu public (actualités, vidéos pédagogiques, back-office) sont terminés côté backend et frontend. Le site tourne en local avec un design validé et des contenus réels servis par l'API. Il reste le forum, le profil RGPD, les finitions et le déploiement sur o2switch.

Point d'attention principal : Docker n'est pas encore installé. La base PostgreSQL, Mailhog (emails de test) et les migrations n'ont donc jamais été exécutés « pour de vrai ». Les tests et l'aperçu tournent sur SQLite. Les 3 migrations ont été écrites à la main et doivent être vérifiées avec `doctrine:migrations:diff` une fois Docker en place. Procédure d'installation : fichier `Procedure-Docker-PokerVitrine.txt` sur le Bureau.

2. Travail réalisé

SPRINT	CONTENU LIVRÉ	ÉTAT
Sprint 0 Fondations	docker-compose (PostgreSQL 16 sur le port 5433, Mailhog, Mercure) ; bundles JWT, CORS, EasyAdmin, Mailer ; entités `User` et `ConsentLog` ; `security.yaml` (argon2id, API sans session) ; clés JWT chiffrées hors Git ; dépôt GitHub.	**Terminé**
Sprint 1 Authentification	**Back :** inscription avec consentement CGU explicite (preuve + IP), mot de passe robuste, email et pseudo uniques ; vérification d'email (lien 24 h à usage unique, renvoi sans révéler les comptes existants) ; connexion JWT 1 h + refresh token 7 j à usage unique ; blocage après 5 échecs/min ; `/api/me` ; 10 tests. **Front :** AuthService, intercepteur JWT avec refresh automatique, AuthGuard ; pages accueil, connexion, inscription, vérification d'email, Mon espace ; design clair + rouge, titres Playfair Display, responsive.	**Terminé**

Sprint 2 Contenu public	**Back :** entités `Article` (slug automatique, brouillon / publication programmée) et `Video` (niveau débutant / intermédiaire / avancé) ; API `/api/articles`, `/api/articles/{slug}`, `/api/videos` (filtres), `/api/videos/{id}`, `/api/stats` ; back-office EasyAdmin sur `/admin` (articles, vidéos, utilisateurs) ; rôles Membre < Modérateur < Admin ; 8 vidéos pédagogiques Kill Tilt et zChance44 en données de démo ; 9 tests. **Front :** pages Actualités (liste paginée, filtre catégorie, détail avec titre et description SEO) et Vidéos (filtres niveau / catégorie, détail) ; lecteur YouTube chargé au clic via youtube-nocookie (aucune requête vers YouTube avant le clic) ; accueil branché sur l'API (carrousel, actualités, chiffres clés réels).	**Terminé**

Décisions prises en cours de route

- **Versions :** Symfony 7.4 et Angular 19 conservés (plus récents que les 6.4 / 18 du cahier des charges, compatibles avec tous les bundles).
- **Design :** thème clair + rouge inspiré de la maquette « Royal Casino », titres Playfair Display ; le visuel central sera fourni par le propriétaire du projet (`frontPoker/public/images/hero.webp`).
- **Vidéos :** contenus pédagogiques des chaînes Kill Tilt et zChance44, intégrées via YouTube.
- **Ajout au modèle :** champ `category` sur les articles (affiché sur les cartes du design).
- **Sauvegardes :** commit + push GitHub à chaque étape validée.

3. Travail restant

SPRINT	À FAIRE	PRIORITÉ
Docker (prérequis)	Installer Docker Desktop, lancer PostgreSQL et Mailhog, exécuter et vérifier les 3 migrations, recharger les données de démo, tester la réception réelle des emails de vérification.	**Dès que possible**
Sprint 3 Forum	Entités ForumCategory / ForumTopic / ForumPost ; lecture publique, écriture réservée aux membres vérifiés ; modération EasyAdmin pour les modérateurs (suppression, épinglage, verrouillage) ; pages catégories, sujets, réponse ; compteur de messages sur l'accueil ; notifications aux abonnés d'un sujet.	**Prochaine étape**
Sprint 4 Profil & RGPD	`GET /api/me/export` (export JSON) ; `DELETE /api/me` avec confirmation par email et anonymisation (pseudo « Utilisateur supprimé », email effacé) ; page profil (pseudo, mot de passe, favoris vidéos) ; tâches planifiées : purge des IP après 90 jours, suppression des comptes inactifs depuis 3 ans ; rédaction des CGU, Politique de confidentialité et Mentions légales ; tests de conformité.	**À faire**
Sprint 5 Finitions & déploiement	Tests Jasmine côté front ; SEO (balises meta dynamiques) ; documentation API (NelmioApiDoc) ; configuration de production ; déploiement o2switch ; sauvegardes PostgreSQL ; guides utilisateur et administrateur.	**À faire**
V2 Boutique	Le premier cahier des charges prévoit une boutique (livres et accessoires, Stripe, factures, CGV, TVA 5,5 % / 20 %) ; le dossier de conception la classe hors MVP. À trancher avant de l'engager. Aussi en V2 : quiz, agenda des tournois, newsletter, double authentification.	**À décider**

4. Déploiement prévu sur o2switch

- Hébergement mutualisé : pas de Docker sur le serveur.
- API Symfony sur un sous-domaine (ex. `api.pokervitrine.fr`) pointant vers `backPoker/public`.
- Front Angular compilé (`ng build`) et servi par Apache, avec une règle `.htaccess` pour les routes.
- Base : PostgreSQL si l'offre le permet (à vérifier dans le cPanel), sinon passage à MySQL.
- Emails via le SMTP o2switch ; tâches RGPD via les crons du cPanel.
- Mercure (temps réel) non utilisable sur un mutualisé : pas de temps réel en production.

5. Prérequis côté propriétaire du projet

ÉLÉMENT	POURQUOI	URGENCE
Docker Desktop	Lancer PostgreSQL et Mailhog, exécuter et vérifier les migrations, recevoir les emails de test.	**Haute**
Accès SSH o2switch	Activer SSH dans le cPanel et y ajouter une clé publique pour permettre le déploiement.	**Moyenne**
Visuel de l'accueil	Image à déposer dans `frontPoker/public/images/hero.webp`.	**Moyenne**
Informations légales	Éditeur, directeur de publication, hébergeur, contact RGPD : nécessaires pour les mentions légales et la politique de confidentialité.	**Sprint 4**
GitHub CLI (facultatif)	Permettre de créer dépôts et pull requests sans passer par le site.	**Basse**

6. Lancer le projet en local

API (port 8000)

```
cd backPoker
php -S localhost:8000 -t public public/index.php
```

Back-office : `http://localhost:8000/admin`

Site (port 4300)

```
cd frontPoker
npx ng serve --port 4300
```

Le port 4200 est occupé par le projet ESCT Boxe.

> **Comptes de démonstration** (base locale uniquement) : `demo@pokervitrine.local` (membre) et `admin@pokervitrine.local` (administrateur), mot de passe `Demo-Poker-2026!`. Tests backend : `php bin/phpunit` dans `backPoker`.